

PyLocalSend 演讲大纲与演讲稿

配套课件： `pylocalsend-presentation.md`
建议总时长：**18~22 分钟**（含演示视频约 2~3 分钟）

整体结构

章节	页码范围	主题	建议时长
开场	第 1 页	项目介绍	0:30
01 PROBLEM	第 2~6 页	问题背景与项目定位	4:00
02 PRODUCT	第 7~12 页	产品功能与使用方式	6:00
03 ARCHITECTURE	第 13~17 页	架构与设计要点	4:30
04 IMPLEMENTATION	第 18~21 页	实现与技术细节	3:00
05 DEMO	第 22~24 页	演示视频与限制	3:00
收尾	第 25~27 页	总结、分工、Q&A	2:00

第 1 页 · 封面

幻灯片标题： `PyLocalSend`

大纲要点

- 自我介绍与项目名称
- 一句话说明项目做什么
- 点出技术栈

演讲稿

大家好，我们今天的展示项目是 **PyLocalSend**，一个面向局域网的文件传输工具。

它的核心思路很简单：发送端在本机登记文件路径，接收端通过命令行或浏览器就能下载；如果老师需要收作业，也可以允许接收端把文件反向上传到发送端电脑。

项目使用 Python 开发，底层是 FastAPI 提供 HTTP 服务，NiceGUI 做 Web 界面，传输过程采用流式读写，尽量降低内存占用。

备注： 语速放慢，给观众 3 秒看清封面副标题。

第 2 页 · 章节页 (01 / PROBLEM)

幻灯片标题： 文件传输的问题，常常不在“能不能传”

大纲要点

- 引出真正痛点：成本、步骤、稳定性
- 过渡到下一页的传统方案

演讲稿

我们先从问题出发。

文件传输这件事，很多时候不是“能不能传”的问题——U 盘能传、网盘能传、聊天软件也能传。真正麻烦的是：**成本高、步骤多、还不稳定。**

接下来我们看几种常见方式各自卡在哪里。

第 3 页 · 传统方式的阻力

幻灯片标题： 传统方式的阻力

大纲要点

- U 盘：硬件依赖、易遗漏

- 网盘：公网中转、隐私与速度
- 聊天软件：体积限制、目录结构差
- 收束到项目目标

演讲稿

第一种是 **U 盘**。它直观，但要反复插拔，多人轮流用时很麻烦，文件夹里漏几个子目录也很常见。

第二种是 **网盘**。文件要先上传到公网，再让对方下载，速度和隐私都不完全在自己手里。

第三种是 **聊天软件**。发小文件还行，大文件经常有限制，而且文件夹结构基本保不住。

所以我们做 PyLocalSend，就是想把“传文件”这件事变简单——在局域网里，少步骤、少中转、尽量可控。

第 4 页 · 传统方案流程图

幻灯片标题：传统方案：先复制到内存

大纲要点

- 结合 SVG 讲“两次搬运”
- 发送端读入内存 → 再发给接收端
- 大文件时内存压力大

演讲稿

这张图是传统方案的典型流程。

发送方先把整个文件读进内存，再通过网络发给接收方，接收方再写入本地。等于文件内容在传输过程中至少被完整处理一遍，**大文件时内存压力很明显**。

如果同一台机器还要同时做别的事，或者文件是几个 GB 的视频、数据集，这种方式就不太友好。

备注：用手指或激光笔沿 SVG 箭头走一遍：发送端 → 内存 → 网络 → 接收端。

第 5 页 · PyLocalSend 流程图

幻灯片标题：PyLocalSend 方案：边读边发

大纲要点

- 对比上一页：流式分片
- 发送端按需读取，不整文件进内存
- 为后面“零拷贝”铺垫

演讲稿

PyLocalSend 的做法不同。

发送端**不会先把整个文件读进内存**，而是按分片从磁盘读取，读一块发一块，接收端边收边写。

你可以把它理解成：文件始终待在原来的磁盘位置上，传输时只是打开一条“水管”，让数据流过去。这也是我们后面会反复提到的**流式传输**思路。

第 6 页 · 项目定位

幻灯片标题：项目定位

大纲要点

- 发送端共享、接收端下载
- 无账号体系、无公网依赖、不提前复制
- 零安装 WebUI、一服务多接收端

演讲稿

项目的定位可以概括成一句话：**发送端负责共享，接收端负责下载。**

我们不搞复杂的账号系统，也不依赖公网网盘中转；发送端只是在本地启动一个 HTTP 服务，把已有文件或文件夹“登记”进来。

接收端这边，打开浏览器链接就能下载，**零安装**；一个发送端可以同时服务多个接收端，用 PIN 或独立的 token 链接区分身份。

第 7 页 · 章节页 (02 / PRODUCT)

幻灯片标题： 两种使用方式

大纲要点

- CLI vs WebUI 分工
- 引出后续功能页

演讲稿

第二部分讲产品怎么用。

PyLocalSend 提供两种入口：**CLI 命令行**适合精确控制和批量操作；**WebUI** 适合课堂演示和零安装接收。两者共用同一套核心服务，只是交互方式不同。

第 8 页 · 发送端 WebUI

幻灯片标题： 发送端 WebUI

大纲要点

- 文件传输 Tab：注册路径、开放勾选
- 接收端管理：token、上传权限、禁用
- 记录与设置

演讲稿

发送端 WebUI 用 `pylocalsend gui` 启动，主要有四个模块。

文件传输里，用户输入本机绝对路径完成注册——注意，这里的“上传”是登记路径，不是把文件复制到项目里。注册后可以勾选哪些路径对接收端开放。

接收端管理里可以创建带 token 的专属链接，复制给同学或另一台电脑；可以按接收端允许或禁止浏览器上传，也可以临时禁用某个链接。

另外还能看下载记录，以及在设置里配置分片大小、浏览器上传的存放目录等。

第 9 页 · CLI 使用

幻灯片标题：CLI 使用

大纲要点

- 适合批量、指定目录、断点续传
- 演示右侧命令（可简略带过）
- `gui` / `upload` / `ls-remote` / `download`

演讲稿

命令行更适合“我知道要下到哪个目录、要下哪些文件”的场景。

右侧是几条典型命令：`gui` 启动管理界面；`upload` 在发送端注册共享路径；`ls-remote` 查看远端可下载列表；`download` 把文件或文件夹下载到指定目录。

CLI 路径支持**并行下载**和 **HTTP Range 断点续传**，传大文件、传很多文件时比浏览器更稳。

备注：若时间紧，只念 `upload` 和 `download` 两句即可。

第 10 页 · 接收端 WebUI（下载）

幻灯片标题：接收端 WebUI

大纲要点

- 单文件：浏览器直接下载
- 文件夹：ZIP 流
- 仅显示已开放项；自动同步

演讲稿

接收端打开 `/r/令牌` 链接就能进 WebUI，不需要再装 Python。

单文件由浏览器直接向发送端拉流，保存到浏览器默认下载目录。

文件夹则由发送端按开放范围实时打包成 ZIP 流，浏览器下载为一个 zip 文件。

页面上只会看到发送端勾选开放过的项目，而且每 2 秒轮询 catalog 版本，发送端改了开放范围，接收端列表会自动刷新，不用手动点刷新。

第 11 页 · 接收端浏览器上传

幻灯片标题：接收端浏览器上传

大纲要点

- 默认禁止，发送端单独允许
- 「上传文件」标签页 + NiceGUI 组件
- 存放路径规则；不自动进共享列表（下一页会再强调）

演讲稿

除了下载，我们还做了**反向上传**，适合收作业、收实验数据这类场景。

默认每个接收端都不能上传。发送端在「接收端管理」里对某个接收端点**允许上传**之后，对方页面会多出一个**「上传文件」**标签，用 NiceGUI 的上传组件选文件，支持多文件自动上传。

文件会落到发送端配置的目录下，路径是 存放根目录/接收端名称/文件名。根目录可以在设置里改，默认在 `~/.pylocalsend/browser_uploads`。同名文件会自动加时间戳，避免覆盖。

这里要特别说明：**上传成功以后，文件不会自动出现在发送端共享列表里**。老师如果想再发给别人下载，需要自己去「文件传输」里手动注册那条路径。

第 12 页 · 下载开放与接收端控制

幻灯片标题：下载开放与接收端控制

大纲要点

- 勾选 = grants，可细到子路径
- 禁用 = 停下载 + 停上传 + 中断进行中下载

- 允许上传 / 全部开放 / 自动同步

演讲稿

这一页把权限控制串起来。

开放勾选决定接收端能看到什么：可以只开放文件夹里的某几个子目录，状态存在 `download_grants` 表里。

禁用接收端是一键熔断：新的下载和上传都不行，已经在进行的下载也会被中断；解除禁用后链接恢复正常。

另外还有按接收端开上传、一键全部开放，以及前面说的 `catalog` 自动同步。整体思路是：共享范围可控、接收端身份可控、传输过程可记录。

第 13 页 · 章节页 (03 / ARCHITECTURE)

幻灯片标题：路径登记，按需读取，流式返回

大纲要点

- 章节过渡
- 强调“从两次上传变成一次上传”

演讲稿

第三部分看架构。

如果用一句话概括 `PyLocalSend` 的设计，就是：**路径登记，按需读取，流式返回**——也就是把传统方案里“先整文件进内存再发”的两次搬运，变成边读边发的一次流水。

第 14 页 · 整体架构

幻灯片标题：整体架构

大纲要点

- 左：发送端（FastAPI、NiceGUI、SQLite、本地文件）
- 右：接收端（CLI、浏览器、下载目录）
- 中间：HTTP 流、PIN/TOKEN、Range、可选 Upload

演讲稿

架构上就是两台电脑通过 HTTP 通信。

发送端跑 SenderService，对外提供 FastAPI 接口；NiceGUI 做管理页和接收端页面；SQLite 存元数据、开放范围和下载日志；真正的文件还在本地磁盘上。

接收端可以是 CLI 客户端，也可以是浏览器 WebUI。

中间这条链路支持流式传输、PIN 和 token 鉴权、Range 断点续传，以及可选的浏览器上传。注意上传是额外能力，默认关闭。

第 15 页 · 文件共享流程

幻灯片标题：文件共享流程

大纲要点

- 六步流程走读
- 三个关键词：不复制、不缓存、低内存

演讲稿

共享一条文件的完整流程分六步：启动服务 → 登记路径 → 元数据写入 SQLite → 接收端拉列表 → 发起下载 → 发送端分片读取并返回。

注册时**不复制**文件，只记路径；下载时**不缓存**整文件到临时目录，而是直接读源文件；读写都按分片进行，所以**内存占用低**，适合大文件。

备注：六格 flow 从左到右快速点完，每格 5~8 秒。

第 16 页 · 文件夹下载策略

幻灯片标题： 文件夹下载策略

大纲要点

- CLI：目录树 + 递归 + 并行/续传
- Web：ZIP 流，不落盘临时包
- 示意图 folder → folder.zip

演讲稿

文件夹下载，CLI 和 Web 策略不同。

CLI 会先拿目录树，然后在接收端按相对路径一层层建目录，支持并行和断点续传，适合完整同步。

WebUI 则把文件夹实时打成 ZIP 流发给浏览器，**不会在发送端先生成一个临时 zip 文件**，省磁盘也省等待。

所以浏览器里你看到的是 `folder.zip`，但发送端磁盘上并没有多出一个压缩包。

第 17 页 · 访问控制与记录

幻灯片标题： 访问控制与记录

大纲要点

- 六条 line-list 各用一句话
- 不必逐条念 API 名称

演讲稿

安全和可观测性靠这几块。

PIN 做发送端统一验证；**TOKEN** 给每个接收端独立链接，可禁用、可管上传；**GRANT** 控制哪些路径可见、可下。

UPLOAD 按接收端开关浏览器上传，存放目录用 `upload_dir` 配置。

LOG 记录谁什么时候下了什么、传了多少字节；**VER** 用 catalog 版本号让 WebUI 自动感知列表变化。

这些能力组合起来，既方便课堂临时共享，也方便事后查记录。

第 18 页 · 章节页 (04 / IMPLEMENTATION)

幻灯片标题：代码结构保持清晰

大纲要点

- core 与入口分离
- 过渡到技术栈

演讲稿

第四部分简单过一下实现。

我们把 CLI 和 GUI 都做成薄入口，**核心业务全部放在 core 层**，这样命令行和网页用的是同一套传输、鉴权、数据库逻辑，避免两套代码各写一遍。

第 19 页 · 技术栈

幻灯片标题：技术栈

大纲要点

- 每项一句话，不必展开安装细节
- FastAPI / NiceGUI / httpx / Rich / SQLite / cryptography

演讲稿

技术选型比较克制。

FastAPI 负责 HTTP API 和流式下载；**NiceGUI** 做发送端和接收端页面；**httpx** 给 CLI 发异步请求；**Rich** 画命令行进度条；**SQLite** 做本地持久化；**cryptography** 提供可选的 AES-CTR 加密

流。

没有引入很重的中间件，整个项目适合作为 Python 网络编程和文件 IO 的课程作业。

第 20 页 · 项目结构

幻灯片标题：项目结构

大纲要点

- cli / core / gui / tests / doc
- core 下各子包职责

演讲稿

目录结构也按职责划分。

cli 是命令入口；core 里是文件处理、接收端客户端、发送端服务、配置和加密；gui 分 sender 和 receiver 两套页面；tests 覆盖 CLI 和下载链路；doc 放课件和演示材料。

如果评委想追代码，建议从 core/transfer/sender.py 和 gui/sender/app.py 看起。

第 21 页 · 数据持久化

幻灯片标题：数据持久化

大纲要点

- ~/.pylocalsend 下三个文件
- 五张表/配置的用途

演讲稿

运行时数据放在用户目录 ~/.pylocalsend。

config.json 存端口、分片、上传目录等；session.json 给 CLI 记连接；pylocalsend.db 是 SQLite。

库里有 **files** 登记共享路径，**download_grants** 记开放范围，**receivers** 记接收端和是否允许上传，**logs** 记下载记录。

再次强调：接收端浏览器上传的文件**默认只写磁盘**，不写进 `files` 表，除非发送端手动注册。

第 22 页 · 章节页 (05 / DEMO)

幻灯片标题： 课堂演示

大纲要点

- 过渡到视频
- 可一句话预告演示内容

演讲稿

第五部分是演示。

接下来用一段录屏展示从启动 GUI、注册文件、创建接收端链接，到浏览器下载和反向上传的完整流程。如果现场网络允许，也可以边放视频边对照操作。

第 23 页 · 演示视频

幻灯片标题： 演示视频

大纲要点

- 播放 `doc/video/demo.mp4`
- 播放时旁白可简化为逐步解说
- 重点画面：勾选开放、token 链接、上传标签、上传后不自动共享

演讲稿（播放时旁白）

现在请大家看视频。

（开始播放）

首先启动 `pylocalsend gui`，在文件传输里注册本机路径.....

勾选要开放的文件或子目录.....

在接收端管理创建链接，复制给另一台电脑或另一个浏览器标签.....

接收端页面只能看到已开放的项目，选择后下载.....

如果允许上传，会出现「上传文件」标签，选文件后文件写到发送端配置目录.....

注意：此时发送端共享列表里还没有这项，需要老师手动注册路径才会开放给别人下载。

（视频结束）

以上就是完整演示。

备注：若视频已含配音，本页可只放视频，演讲者不说话或只补一句“细节见视频”。

第 24 页 · 当前限制

幻灯片标题：当前限制

大纲要点

- 浏览器不能指定任意保存路径
- 默认面向可信局域网
- 上传不自动共享；大文件 Web 上传有内存限制
- CLI vs WebUI 适用场景

演讲稿

诚实说一下当前限制。

第一，WebUI 下载位置由浏览器决定，不能像 CLI 那样写到任意绝对路径，这是浏览器安全模型决定的。

第二，项目默认面向**可信局域网**，要上公网得自己做端口映射和安全加固。

第三，接收端上传**不会自动共享**；第四，Web 上传要把文件经浏览器读入内存，超大文件更适合 CLI 或本机直接注册路径。

所以我们的结论是：CLI 适合复杂批量下载，WebUI 适合零安装接收、反向上传和课堂演示。

第 25 页 · 总结 (SUMMARY)

幻灯片标题：PyLocalSend

大纲要点

- 复述核心价值
- 六个关键字各点一下

演讲稿

最后总结一下。

PyLocalSend 让发送端只登记路径，下载时流式读取，尽量降低操作复杂度。

六个关键词：**低成本**——局域网直连；**局域网**——不依赖公网中转；**流式传输**——边读边发；**可管理**——接收端、记录、禁用都可控；**细粒度开放**——勾到哪下到哪；**反向上传**——按需收文件到发送端。

希望这个工具在课堂、实验室里能真正省几步操作。

第 26 页 · 分工说明

幻灯片内容：李子健 / 杨秉熹 分工列表

大纲要点

- 简短介绍两人分工
- 不必展开技术细节

演讲稿

项目由两人协作完成。

李子健负责 GUI 和 CLI 的设计与实现、核心逻辑代码，以及演示视频的剪辑。

杨秉熹负责核心逻辑代码、视频素材提供和 PPT 制作。

核心业务我们是一起打磨的，入口层和文档按特长做了分工。

第 27 页 · Q&A

幻灯片标题：Q&A

大纲要点

- 邀请提问
- 准备 2~3 个常见问题备用

演讲稿

我们的展示到这里结束，谢谢大家。

欢迎提问。如果被问到下面几类，可以参考回答：

问：和网盘、飞书传文件有什么区别？

答：我们不做公网存储，数据在局域网直连，发送端完全掌握共享范围和接收端权限。

问：为什么上传不自动共享？

答：故意分开两条链路——先收到老师电脑，再由老师决定要不要、对谁开放，避免学生误传文件直接暴露给所有人。

问：大文件怎么办？

答：下载走流式分片；CLI 支持断点续传。上传若很大，建议本机注册路径或后续扩展分片上传。

备注：Q&A 页可停留，分工若已在上页说完，本页只保留“谢谢观看 + 欢迎提问”。

附录：演讲节奏建议

1. **前 6 页（问题 + 定位）** 不要太快，把“为什么做”讲清楚，评委先建立共鸣。
2. **第 4~5 页两张流程图** 是对比高潮，适当停顿，让观众看清箭头方向。
3. **第 11 页上传** 是功能亮点，务必讲清“上传 ≠ 自动共享”。
4. **第 23 页视频** 可静音播放 + 现场解说，或原声播放 + 演讲者少说话。

5. **控制总时长：**若只有 15 分钟，可压缩第 19~21 页（技术栈/结构/数据库）各 30 秒，把省下的时间留给视频。

附录：双人配合建议（可选）

段落	建议主讲
第 1~6 页 背景与定位	杨秉熹（PPT）
第 7~12 页 产品功能	李子健（GUI/CLI 实现者）
第 13~17 页 架构	两人交替，李子健讲流程图，杨秉熹讲控制与记录
第 18~21 页 实现	李子健
第 22~24 页 演示与限制	李子健播视频，杨秉熹补限制说明
第 25~27 页 收尾	杨秉熹总结 + 两人一起 Q&A